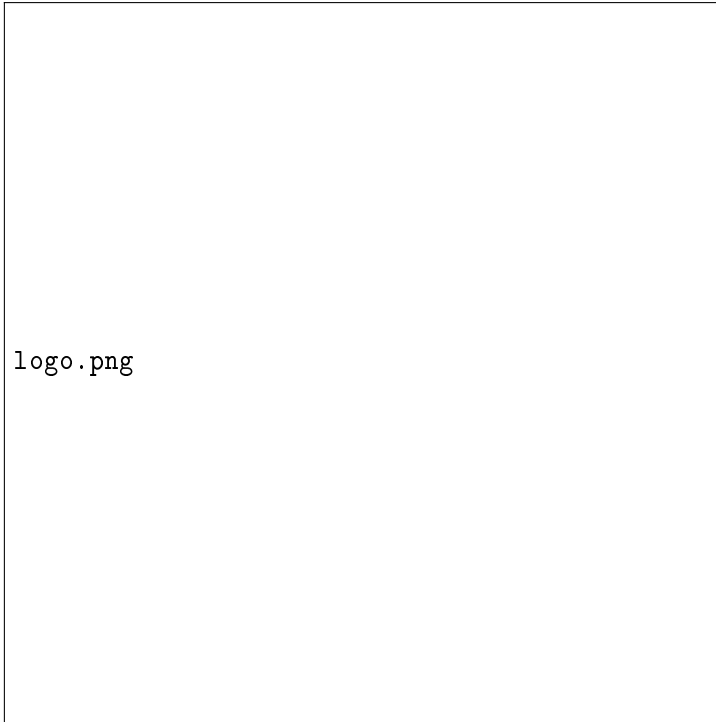




CMP6228

Deep Neural Networks

Assessment 2 – Final Report

**Comparative Transfer Learning for Blood Cell
Classification**

**with a Data-Integrity Audit and Grad-CAM
Explainability**

Student Name: Shekhar Lamichhane Magar
Student ID: 23189647
Course: Deep Neural Networks
Assessment: Assessment 2 – Final Report
Word Count: ~3,000 (main text, Sections 1–5)
Academic Year: 2025–26

Abstract

This report investigates transfer learning for classifying peripheral blood cells into three classes — red blood cells (RBCs), white blood cells (WBCs), and platelets — using microscopy crops derived from the TXL-PBC dataset. Three ImageNet-pretrained backbones, EfficientNet-B0, MobileNetV3-Small, and DenseNet121, are compared under an identical training configuration, with class imbalance addressed through a weighted sampler and weighted cross-entropy loss. An initial evaluation gave EfficientNet-B0 perfect test accuracy, which prompted a data-integrity audit: filename, source-image, and official split-list checks found no conventional leakage, but a perceptual-hash audit flagged 694 crops (675 of them RBCs) as near-duplicates, 113 of them in the test split. All three models were retrained and evaluated on the cleaned dataset (1,768 test crops). EfficientNet-B0’s perfect score did not survive (accuracy 0.997172, macro-F1 0.989587), and MobileNetV3-Small became the best model (accuracy 0.999434, macro-F1 0.995184) while also training fastest, ahead of DenseNet121 (0.998303 and 0.992360). The models differ by at most four test crops from single runs, so the new ranking is not statistically established and residual leakage cannot be excluded. Grad-CAM overlays are provided as qualitative examples.

Contents

1	Introduction	5
2	Problem Statement	5
2.1	Dataset	5
2.2	Problem Formulation	6
2.3	Related Work and Recent Trends	6
3	Proposed Method	6
3.1	Preprocessing and Data Preparation	6
3.2	Near-Duplicate Audit and Dataset Cleaning	6
3.3	Model Architectures	7
3.4	Loss Function and Class Imbalance	7
3.5	Training and Model Selection	8
4	Experimental Results	8
4.1	Experimental Setup	8
4.2	Model Comparison	8
4.3	Status of the Perfect-Accuracy Result	10
4.4	Grad-CAM Explainability	10
4.5	Discussion	11
5	Summary	11
5.1	Study Overview	11
5.2	Main Findings	11
5.3	Data-Integrity Findings	12
5.4	Explainability	12
5.5	Limitations	12
5.6	Future Work	12
5.7	Practical and Ethical Considerations	12
	References	13
	Appendices	13
A	Per-Model Confusion Matrices	13
B	Near-Duplicate Audit Evidence	14
C	Reproducing the Pipeline	16

List of Figures

1	Pipeline	7
2	Confusion matrices, cleaned test set	9
3	Grad-CAM examples	11
4	EfficientNet-B0 confusion matrix (cleaned test set).	13
5	MobileNetV3-Small confusion matrix (cleaned test set).	13
6	DenseNet121 confusion matrix (cleaned test set).	14
7	Example flagged crop pairs	14
8	Sample test crops by class	15
9	Example flagged pairs between splits	15

List of Tables

1	Crops per split before near-duplicate cleanup	5
2	Settings used for all experiments.	8
3	Cleaned test-set results	8
4	Before and after cleaning	10
5	Contents of the code archive.	16

1 Introduction

Blood cell analysis and counting support clinical diagnosis (?), and automated classification of red blood cells (RBCs), white blood cells (WBCs), and platelets could reduce the manual effort of reviewing microscopy fields. This report applies transfer learning to this three-class problem using TXL-PBC (?), a public collection that integrates four peripheral blood smear datasets, and addresses two practical challenges: severe class imbalance and an initially suspicious evaluation result.

All three models scored unusually high at first, EfficientNet-B0 reaching 100% test accuracy. Rather than report this uncritically, the data were audited for leakage, which flagged near-duplicate crops. A cleaned dataset was built and all three models were retrained and re-evaluated on it, so the effect of cleaning could be measured directly.

The aim is to compare three pretrained backbones and to establish how far the results can be trusted. The contributions are:

- an imbalance-aware comparison of EfficientNet-B0, MobileNetV3-Small, and DenseNet121 before and after near-duplicate removal;
- a data-integrity audit that flagged 694 near-duplicate crops, with an assessment of its limits and of its effect on the results;
- a qualitative Grad-CAM analysis of a pre-cleanup EfficientNet-B0 checkpoint.

2 Problem Statement

2.1 Dataset

TXL-PBC (?) is a curated collection integrating four public resources (BCCD, BCDD, PBC and Raabin-WBC), released under the MIT licence. It contains 1,260 smear images with 18,143 annotated cells, each image having a YOLO-format file of bounding boxes and class indices (0 = WBC, 1 = RBC, 2 = Platelet). Each box is cropped and written to per-class folders under `train/`, `val/` and `test/`.

Splitting uses TXL-PBC’s official source-image lists (882 training, 252 validation and 126 test images; about 70/20/10) rather than a random split, which keeps results comparable with other work and avoids crops of one image appearing in several splits. Table 1 gives the crop counts before near-duplicate cleanup. RBCs outnumber platelets by roughly 35:1 in the original test set and 29:1 in training (33:1 and 28:1 after cleaning), which motivates macro-averaged metrics and imbalance-aware training.

Table 1: Crops per split before near-duplicate cleanup, with crops flagged for removal by the audit (Section 3.2) in brackets.

Split	WBC	RBC	Platelet	Total
Train	908 (7)	11,220 (366)	382 (0)	12,510 (373)
Val	257 (5)	3,383 (203)	112 (0)	3,752 (208)
Test	133 (6)	1,699 (106)	49 (1)	1,881 (113)
Total	1,298 (18)	16,302 (675)	543 (1)	18,143 (694)

2.2 Problem Formulation

Given a cropped cell image $x \in \mathbb{R}^{H \times W \times 3}$, the model outputs a probability distribution over $C = 3$ classes:

$$\hat{y} = \text{softmax}(f_{\theta}(x)) \in \mathbb{R}^C.$$

Because the classes are skewed, macro-F1 is the primary metric alongside accuracy, since accuracy can be dominated by RBC performance.

2.3 Related Work and Recent Trends

Blood cell classification is commonly studied on datasets of individual cropped cells. The PBC dataset (?), with thousands of images in eight cell groups, is widely used and is one of the four resources integrated into TXL-PBC, which adds bounding boxes for the three classes used here.

Recent work has moved towards vision transformers (?) and modernised ConvNets such as ConvNeXt (?), typically pretrained at scale; these were not evaluated. Compact CNNs suit a three-class task on a modest dataset, and earlier work reports that much smaller architectures can match standard ImageNet models on medical tasks (?). MobileNetV3 (?) targets resource-constrained devices, making it a natural efficiency baseline.

Evaluation validity is a further concern. ? found near-duplicates between CIFAR training and test sets, ? document leakage as a common source of over-optimistic results, and ? describe shortcut learning. ? show that some saliency methods are insensitive to model parameters, so interpretability also needs sanity checks. These points motivate the audit in Section 3.2 and the caution in Section 4.3.

3 Proposed Method

3.1 Preprocessing and Data Preparation

All experiments use PyTorch (?); dependencies are in `requirements.txt`, and the full source is in `code.zip` (Appendix C). `crop_dataset.py` converts each YOLO box to pixel coordinates, discards degenerate crops and writes the class folders, using the official split lists (a random 70/15/15 fallback exists but was not used). Augmentation and the `WeightedRandomSampler` are listed in Table 2. Offline oversampling is configured but was not executed, so class balance relies on the sampler and loss weights alone.

3.2 Near-Duplicate Audit and Dataset Cleaning

The initial 100% test accuracy was treated as a signal to investigate. Three checks were run. First, filenames and source-image identifiers were compared across splits, and second, the official split lists were checked for repeated filenames; no overlap was found in either. Third, a perceptual hash (pHash; the default 64-bit `imagehash` hash) was computed for every crop and compared pairwise across splits, only between crops of the same class, with a Hamming-distance threshold of 5. No sensitivity analysis was run for this threshold.

The pHash audit found near-duplicate pairs in every split combination: 382 between train and validation, 168 between train and test and 42 between validation and test (592 in total, 582 involving RBCs). In total 694 crops were flagged for removal (Table 1), keeping the first crop of each near-duplicate group in train, validation, test order, so a duplicate in an evaluation split is removed rather than the training copy. The test set lost 113 of its 1,881 crops (6.0%). By class, 4.1% of RBCs, 1.4% of WBCs and 0.2% of platelets were flagged. Because 694 exceeds 592, some removals reflect within-split duplicates; the breakdown was not recorded.

Cleaned dataset and retraining. The flagged crops were removed, leaving 12,137 training, 3,544 validation and 1,768 test crops (test: 127 WBC, 1,593 RBC, 48 platelet). All three models were retrained from their ImageNet initialisation and re-evaluated with the settings in Table 2; the pre-cleanup checkpoints and results were preserved separately. The two evaluations therefore use different test sets (1,881 and 1,768 crops).

The audit has limits. pHash is not invariant to flips or rotations. Comparisons were within class only, so near-identical crops with different labels are not flagged; conversely, several example pairs in Appendix B differ visibly in staining or shape, so 694 is best read as an upper bound. Duplicates across splits despite distinct filenames may reflect overlap between the four integrated collections, but this was not tested, and hashing was done on crops rather than whole images.

3.3 Model Architectures

Three ImageNet-pretrained (?) backbones are compared (Figure 1): EfficientNet-B0 (?), the primary model, for its accuracy-to-size ratio; MobileNetV3-Small (?) as an efficiency baseline; and DenseNet121 (?) as a feature-reuse baseline. Each ImageNet classifier is replaced by a three-class head (Table 2). Trainable parameters are approximately 4.0, 1.5 and 7.0 million respectively, derived from the standard torchvision architectures rather than measured from the checkpoints.

Figure 1: Pipeline overview. The post-cleanup runs use the cleaned crops; the pre-cleanup runs skip the “Cleaned crops” step. Macro-AUC is reported only for the post-cleanup runs.

3.4 Loss Function and Class Imbalance

Imbalance is handled twice. The sampler balances mini-batches, and the loss is cross-entropy with effective-number class weights (?),

$$\mathcal{L}_{\text{WCE}} = -\frac{1}{N} \sum_{i=1}^N w_{y_i} \log \hat{p}_{i,y_i}, \quad w_c \propto \frac{1 - \beta}{1 - \beta^{n_c}}, \quad \beta = 0.999,$$

where n_c is the number of training crops in class c in the dataset being used, normalised so weights average about one. Focal loss (?) was implemented but not used. Because both mechanisms compensate for imbalance, minority classes are effectively corrected twice, and their separate contributions were not ablated.

Table 2: Settings used for all experiments.

Setting	Value
Backbones	EfficientNet-B0, MobileNetV3-Small, DenseNet121; torchvision IMAGENET1K_V1 weights; all layers fine-tuned
Input	224×224 , ImageNet normalisation
Head	Dropout 0.3 + linear (3 classes) for EfficientNet-B0 and DenseNet121; linear (3 classes) for MobileNetV3-Small
Loss and sampler	Effective-number weighted cross-entropy ($\beta = 0.999$); WeightedRandomSampler (inverse class frequency)
Optimiser	Adam, learning rate 3×10^{-4} , weight decay 1×10^{-5} , no schedule
Batches and epochs	Batch size 32, maximum 30 epochs
Stopping and selection	Patience 6 on validation macro-F1; best validation macro-F1 checkpoint
Augmentation	Rotation up to 20° , horizontal flip ($p = 0.5$), brightness and contrast jitter (0.2)
Data and seed	Original training split (12,510 crops) or cleaned training split (12,137 crops); seed 42

3.5 Training and Model Selection

Models are fine-tuned with Adam (?) (Table 2). Training stops when validation macro-F1 has not improved for six epochs, and the checkpoint with the best validation macro-F1 is kept, since accuracy alone would favour the majority RBC class.

4 Experimental Results

4.1 Experimental Setup

Table 2 lists the settings shared by all three backbones and both dataset versions. The test set is used only for the final evaluation, and each result is a single run.

4.2 Model Comparison

Table 3 reports results on the cleaned test set (1,768 crops: 127 WBC, 1,593 RBC, 48 platelet). Error counts are implied by accuracy, and macro-AUC is one-vs-rest. Training time is the wall-clock time of a run, which also depends on when early stopping triggered, and evaluation time includes data loading, so neither is a latency benchmark.

Table 3: Cleaned test-set results (1,768 crops), with training and evaluation time in seconds.

Model	Accuracy	Macro-F1	Macro-AUC	Errors	Train (s)	Eval (s)
EfficientNet-B0	0.997172	0.989587	0.999987	5	1,396.73	10.58
MobileNetV3-Small	0.999434	0.995184	1.000000	1	895.60	8.70
DenseNet121	0.998303	0.992360	0.999996	3	1,448.15	12.14

MobileNetV3-Small was the best model (one error), followed by DenseNet121 (three) and EfficientNet-B0 (five). Macro-AUC is close to 1 for all three and does not discriminate between them. Compared with the pre-cleanup results (Table 4), EfficientNet-B0 fell from a perfect

score to five errors, MobileNetV3-Small improved slightly and DenseNet121 changed little, so cleaning reversed the order of the models. MobileNetV3-Small also took about 36–38% less time to train than the other two and had the shortest evaluation time. The confusion matrices (Figure 2) show that all three models misclassified one platelet as a WBC, and EfficientNet-B0 and DenseNet121 each also misclassified one WBC as an RBC. The remaining errors are not visible after rounding to two decimals in the normalised matrices, so they are inferred to be RBC errors; this attribution comes from the rounded matrices, not raw counts.



Figure 2: Normalised confusion matrices (rows sum to 1) on the cleaned test set (1,768 crops). Larger versions are in Appendix A.

Table 4: Pre-cleanup (1,881 test crops) and post-cleanup (1,768 test crops) results.

Model	Accuracy before	Accuracy after	Macro-F1 before	Macro-F1 after
EfficientNet-B0	1.000000	0.997172	1.000000	0.989587
MobileNetV3-Small	0.998937	0.999434	0.993959	0.995184
DenseNet121	0.998937	0.998303	0.993978	0.992360

4.3 Status of the Perfect-Accuracy Result

The perfect score did not survive cleaning: retrained on the cleaned data, EfficientNet-B0 made five errors on 1,768 test crops. This is consistent with near-duplicates having inflated the pre-cleanup score, since the 113 removed test crops included at least some copies of training crops. It does not prove it, because each result is a single run and the training and test sets changed together, so retraining variation is confounded with the effect of cleaning. Cleaning also did not hurt every model (MobileNetV3-Small improved, DenseNet121 changed little), which fits leakage benefiting models unevenly, or run-to-run noise.

Differences between models are statistically small. Exact 95% Clopper–Pearson intervals for the error rate are 0.09–0.66% for EfficientNet-B0 (five errors), 0.001–0.31% for MobileNetV3-Small (one) and 0.04–0.50% for DenseNet121 (three). They overlap, so the new ranking is not established, and the audit’s limits (Section 3.2) mean the cleaned test set may still contain undetected duplicates. Three possible shortcuts were not tested: size or blur cues (small crops such as platelets are enlarged more and appear blurrier, Figure 8), image source (the four collections were acquired under different conditions), and prediction confidence (per-sample softmax outputs were not saved).

4.4 Grad-CAM Explainability

Grad-CAM (?) was applied to the final convolutional layer of the pre-cleanup EfficientNet-B0 checkpoint. In Figure 3, high activation appears to fall on the nucleus for the WBC, the cell body for the RBC and the granular cluster for the platelet, not on the background. Each panel is the first test crop of its class in filename order (not chosen for representativeness), so the figure illustrates rather than establishes this behaviour. Crops are centred, so central attention is expected anyway; the maps are coarse and no saliency sanity check (?) was run. Grad-CAM was generated only for this checkpoint, which is no longer the best model, so these maps say nothing about why any model made an error.

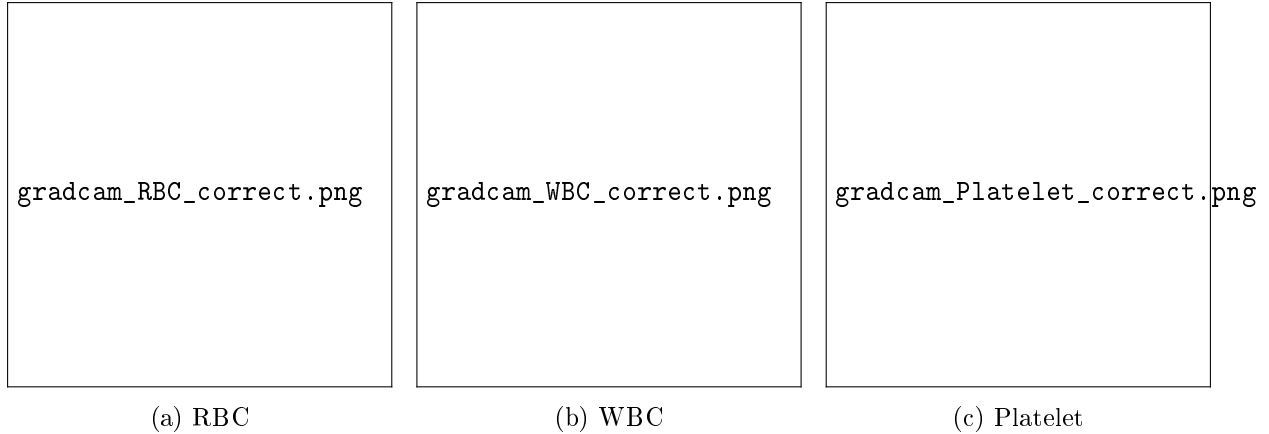


Figure 3: Qualitative Grad-CAM examples (pre-cleanup EfficientNet-B0), one correctly classified crop per class.

4.5 Discussion

After cleaning, MobileNetV3-Small was the best model, with one error, about 38% of EfficientNet-B0’s parameters and the shortest training and evaluation times, so on this dataset the smallest model was also the most accurate and efficient. EfficientNet-B0, the primary model, went from perfect to last, and DenseNet121’s larger size brought no gain. The models differ by at most four crops from single runs, so these orderings are provisional and a different seed could reorder them. Timings are single-run wall-clock values, not a controlled latency benchmark.

5 Summary

5.1 Study Overview

This report compared EfficientNet-B0, MobileNetV3-Small and DenseNet121 for three-class blood cell classification on TXL-PBC under imbalance-aware training, and treated an initial perfect test score as a prompt for investigation rather than a result to report as-is. Filename, source-image and split-list checks found no conventional leakage, but a perceptual-hash audit flagged 694 near-duplicate crops, and all three models were retrained and re-evaluated on the cleaned dataset so the effect of the cleaning could be measured directly rather than assumed.

5.2 Main Findings

On the cleaned test set (1,768 crops), MobileNetV3-Small was the best model (accuracy 0.999434, macro-F1 0.995184) and also the fastest to train (895.60 s), ahead of DenseNet121 (0.998303, 0.992360) and EfficientNet-B0 (0.997172, 0.989587). EfficientNet-B0’s pre-cleanup perfect score did not survive retraining on the cleaned data, and cleaning reversed the model ranking: the smallest model tested became the most accurate. Differences of one to four errors between models fall within overlapping 95% confidence intervals, so this ranking is indicative rather than statistically established.

5.3 Data-Integrity Findings

The near-duplicate audit is the report’s central methodological result: 694 crops (675 of them RBCs) were flagged across every split-pair combination, 113 of them in the test split, despite the dataset using official source-image-level splits. This shows that a split at the image level does not guarantee crop-level independence when the underlying collections themselves may overlap, and that an unusually strong result is itself evidence worth checking before it is reported.

5.4 Explainability

Grad-CAM overlays on the pre-cleanup EfficientNet-B0 checkpoint placed high activation on plausible regions (the nucleus for WBCs, the cell body for RBCs, the granular cluster for platelets) rather than on the background, offering a qualitative, illustrative check consistent with the model attending to the cell rather than to artefacts. No saliency sanity check was run and no map was produced for the retrained, better-performing models, so this evidence is suggestive only.

5.5 Limitations

Every result is a single run at a fixed seed, with error-rate confidence intervals that overlap across models. The pHash audit cannot detect flipped or rotated duplicates and compared crops within class only, so some residual leakage in the cleaned test set cannot be ruled out, and the pre- and post-cleanup evaluations use different test sets, which complicates direct comparison. Grad-CAM was applied to one checkpoint that is no longer the best model, and training/evaluation timings are single-run wall-clock figures rather than a controlled benchmark.

5.6 Future Work

Repeating each model across several seeds, or using grouped cross-validation, would let the current ranking be tested statistically rather than read off single runs. Hashing at source-image level (rather than per-crop) and auditing for leakage before the first training run, rather than after an unusually strong result, would avoid a full retraining cycle and give the split process the earlier scrutiny it needed here. Re-running Grad-CAM with a parameter-randomisation sanity check on the retrained models, saving per-sample predictions, and testing on an independent microscopy dataset would each add evidence this report currently lacks.

5.7 Practical and Ethical Considerations

This is an educational exercise on a public dataset, not a clinical tool. Misclassifying platelets as white cells, or the reverse, could distort automated cell counts, and any real diagnostic use would require external validation on independent patient data, ethical approval, and review by a qualified clinician. Because the four collections integrated into TXL-PBC were acquired under different conditions, results here should not be assumed to transfer to other scanners, staining protocols or patient populations without further testing.

A Per-Model Confusion Matrices

Normalised confusion matrices (rows sum to 1) for each model on the cleaned test set (1,768 crops), from the post-cleanup evaluation.

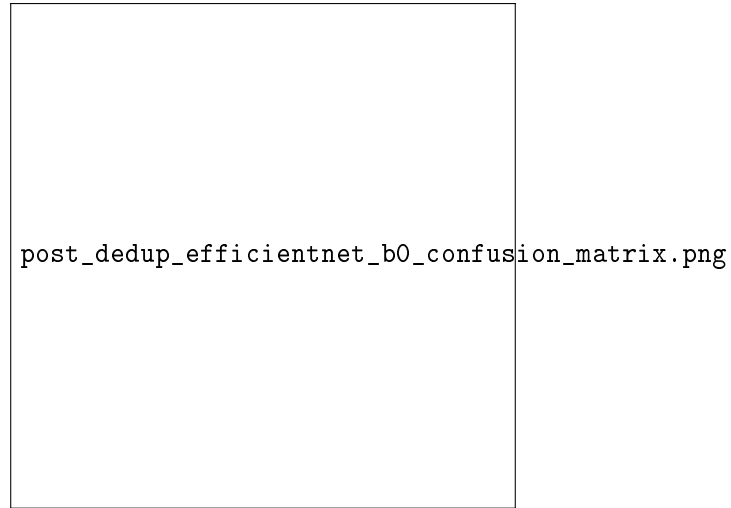


Figure 4: EfficientNet-B0 confusion matrix (cleaned test set).

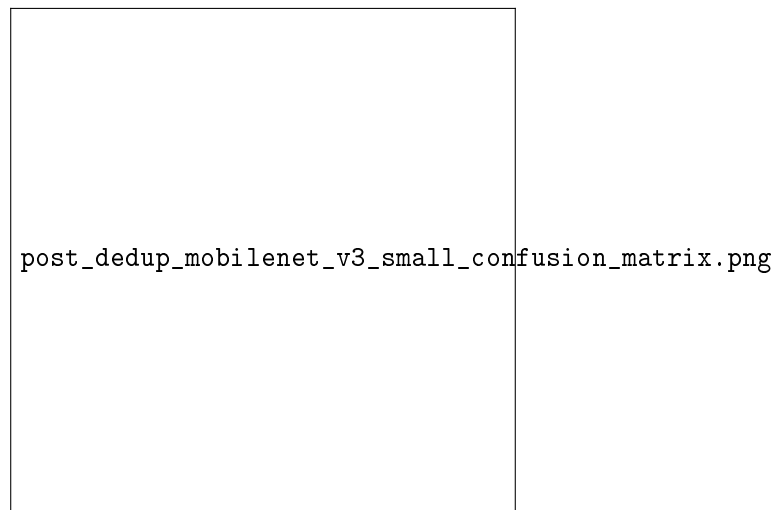


Figure 5: MobileNetV3-Small confusion matrix (cleaned test set).

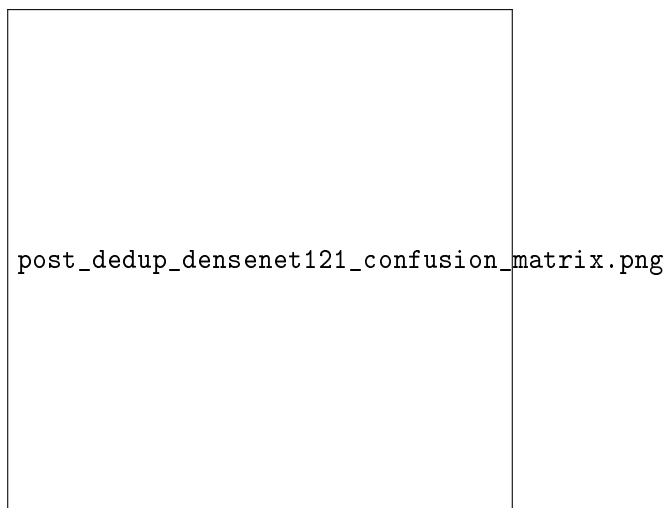


Figure 6: DenseNet121 confusion matrix (cleaned test set).

B Near-Duplicate Audit Evidence

The figures below show example pairs flagged by the perceptual-hash audit. Several pairs, particularly in the train–test and validation–test panels, differ visibly in staining or shape, so some flags are probably false positives. The Hamming distance of each pair is not annotated.

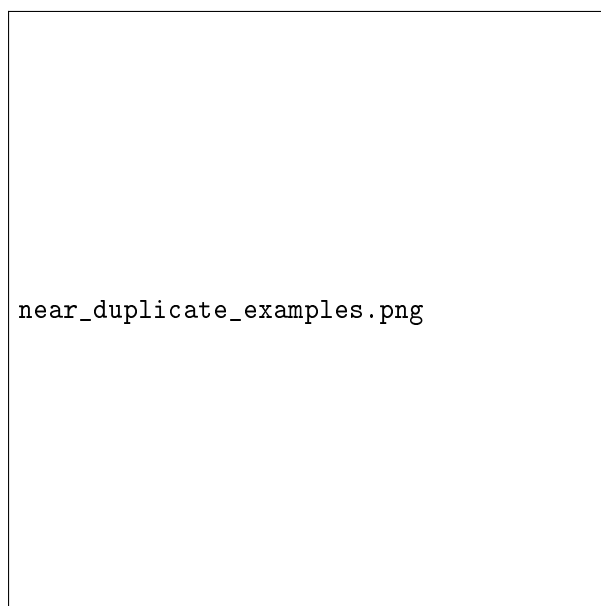
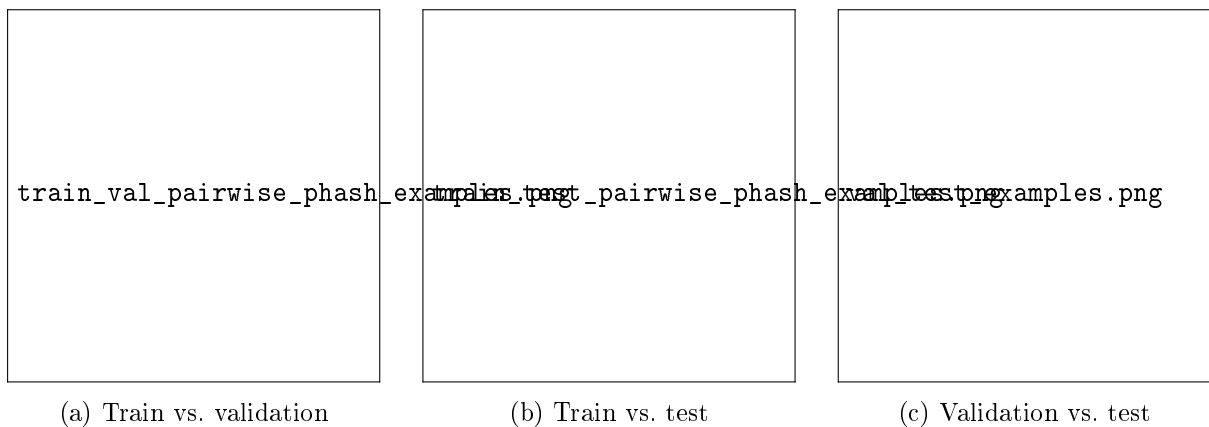


Figure 7: Example near-duplicate crop pairs flagged by the perceptual-hash audit.



Figure 8: Sanity-check sample grid of test crops by class. The platelet crops appear smaller and blurrier than the other classes.



(a) Train vs. validation

(b) Train vs. test

(c) Validation vs. test

Figure 9: Example near-duplicate pairs flagged between splits; the two crops in each row are one flagged pair.

C Reproducing the Pipeline

The full project, including the configuration file and requirements, is supplied as `code.zip` with this report (Table 5); source listings are not reproduced here. Results before and after cleaning were written to separate folders (`outputs_pre_dedup/` and `outputs_post_dedup/`). The commands below reproduce the pipeline.

Table 5: Contents of the code archive.

File	Purpose
<code>src/crop_dataset.py</code>	Crops annotated cells and builds the class folders
<code>src/deduplicate_processed_dataset.py</code>	pHash near-duplicate audit and removal
<code>src/train.py</code>	Model construction, sampler, loss and training loop
<code>src/evaluate.py</code>	Test-set evaluation and metrics
<code>src/plot_model_results.py</code>	Comparison figures
<code>src/gradcam.py</code>	Grad-CAM overlays
<code>configs/config.yaml</code>	Hyperparameters and paths
<code>requirements.txt</code>	Python dependencies

Listing 1: End-to-end commands

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Build cropped dataset from raw images + YOLO labels
python src/crop_dataset.py --config configs/config.yaml

# 3. Audit near-duplicate crops (dry run; writes the audit summary)
python src/deduplicate_processed_dataset.py

# 4. Train and evaluate on the original splits (pre-cleanup results)
python src/train.py --config configs/config.yaml --model efficientnet_b0
python src/train.py --config configs/config.yaml --model mobilenet_v3_small
python src/train.py --config configs/config.yaml --model densenet121
python src/evaluate.py --config configs/config.yaml --model all
python src/plot_model_results.py --config configs/config.yaml

# 5. Remove the flagged crops, then repeat step 4 with the output
#     folders set to a separate location (post-cleanup results)
python src/deduplicate_processed_dataset.py --apply

# 6. Generate Grad-CAM overlays
python src/gradcam.py --config configs/config.yaml --model efficientnet_b0
```